

Sorbonne Université



Analyse d'Algorithmes, Graphes et Applications

Rapport de projet

Ordonnancement pour l'énumération efficace de triangles

Mickaël VILAYVANH
Elise LOUIS

Novembre 2025

Introduction

Le présent rapport aborde la problématique de l'ordonnancement efficace des triangles dans un graphe orienté.

L'objectif est de concevoir et d'évaluer des heuristiques plus performantes que l'ordonnancement par dégénérescence, dans le but de réduire le coût computationnel de la recherche de triangles, notamment sur des graphes de tailles, de structures et de densités variées, parfois très complexes.

Notre approche repose sur l'étude d'heuristiques fondées sur différentes propriétés structurelles des graphes, telles que l'homophilie, le clustering, le degré et la centralité, appliquées ensuite aux algorithmes de recherche de triangles à la pointe (state-of-the-art) A++ et A+-. décrits dans l'article [Tailored vertex ordering for faster triangle listing in large graphs](#).

Vous trouverez, par ailleurs, le dépôt du projet à l'adresse:

<https://github.com/mVilayvanh/AAGA-Ordonnancement-enumeration-triangle>.

Conception, structure et implémentation des heuristiques

Dans cette partie, nous allons vous détailler chaque étape de conception des heuristiques.

1. Structures

a. Graphe orienté

Pour représenter un graphe orienté, nous avons opté pour une liste d'adjacence implémentée sous la forme d'un dictionnaire : les clés correspondent aux nœuds du graphe et les valeurs sont les listes de leurs successeurs.

Dans cette optique, nous avons défini une classe `DirectedGraph` possédant un attribut destiné à stocker ce dictionnaire principal.

Afin d'optimiser les performances lors de l'exécution des algorithmes, nous avons également mis en place des dictionnaires auxiliaires qui se mettent à jour automatiquement à chaque ajout d'arête.

Cette approche permet d'éviter des recalculs coûteux au moment de l'ordonnancement et de garantir un accès en temps constant ($O(1)$) aux informations relatives à un nœud.

Plus précisément, un dictionnaire supplémentaire est utilisé pour connaître les successeurs de chaque nœud, et un autre pour identifier ses prédécesseurs, ainsi qu'un dictionnaire pour connaître le degré de chaque nœud. Cela est nécessaire pour effectuer les deux algorithmes A++ et A+-.

b. Algorithme A++

Cet algorithme correspond au parcours L+n de l'algorithme K3 qui permet d'énumérer les triangles orientés d'un graphe orienté. Pour l'algorithme A++ et A+-, nous considérons ce triangle provenant de l'article:

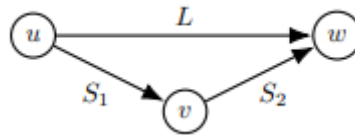


Figure 1. Schéma d'un triangle orienté

Dans la variante A++, il faut démarrer l'évaluation d'un triangle en partant du nœud w . Ensuite, on marque ces prédécesseurs pour les garder en mémoire, ici u et v et pour chaque prédécesseurs, on vérifie s'il existe un prédécesseur de celui-ci qui appartient aussi aux prédécesseurs du nœud de départ. C'est le cas par exemple en démarrant de w et en prenant le prédécesseur v . u est prédécesseur de ces deux nœuds donc il constitue un triangle orienté. On réitère ce principe sur tous les nœuds du graphe en faisant bien attention de réinitialiser les marquages pour la prochaine itération. Pour faire simple, pour un nœud u et on marque tous ses prédécesseurs. Si un prédécesseur v du nœud u possède un prédécesseur w marqué car il est aussi prédécesseur de u , alors c'est un triangle orienté.

c. Algorithme A+-

Comme dans le paragraphe précédent, nous nous basons sur le même schéma de triangle présenté dans la Figure 1. L'objectif ici est d'évaluer un même triangle orienté, mais selon un parcours différent. Le principe de l'algorithme A+- correspond à la variante S1+n du modèle K3.

Sur le schéma, on part du nœud u . Dans un premier temps, tous les successeurs de u sont marqués : dans notre exemple, v et w . Ensuite, pour chacun de ces successeurs, on vérifie si l'un de leurs successeurs est également un successeur du nœud de départ u .

Autrement dit, si un successeur v de u possède un successeur w déjà marqué (c'est-à-dire également successeur de u), alors un triangle orienté est détecté. Dans la Figure 1, ce cas est illustré lorsque, à partir de u , on explore le successeur v : le nœud w , successeur à la fois de u et de v , permet d'identifier un triangle.

Pour l'ensemble des tests et expérimentations présentés dans ce rapport, nous nous référerons désormais exclusivement à la version A+- de l'algorithme.

d. Ordonnement par dégénérescence

Pour calculer l'ordre de dégénérescence d'un graphe orienté à partir des degrés entrants de ses nœuds. L'algorithme commence par déterminer les degrés entrants de chaque sommet et les stocke dans un dictionnaire.

Ces valeurs sont ensuite insérées dans une file de priorité (min-heap) afin de permettre l'extraction successive du nœud ayant le plus petit degré entrant. La fonction parcourt ensuite le graphe en retirant à chaque itération le nœud de plus faible degré non encore visité, qu'elle ajoute à la liste d'ordre.

Après chaque suppression, les degrés des successeurs du nœud traité sont décrémentés et la structure de tas est mise à jour pour refléter ces changements. Cette approche garantit que

l'ordonnancement produit respecte la logique du calcul de dégénérescence, en traitant les sommets selon une progression du degré minimal.

La fonction retourne finalement la liste ordonnée des nœuds, correspondant à l'ordre de dégénérescence du graphe.

2. Intuitions

Pour rechercher des heuristiques efficaces en matière d'ordonnancement, nous nous sommes intéressés à plusieurs notions fondamentales issues de la théorie des graphes, notamment le degré, l'homophilie, la centralité, le PageRank et la détection de communautés.

L'objectif étant d'améliorer l'efficacité de la recherche de triangles dans les graphes, nous avons concentré notre attention sur des types d'ordonnancement présentant un intérêt ou un lien potentiel avec ces propriétés structurelles.

Pour commencer intéressons nous au degré des nœuds d'un graphe, une heuristique qui avait été proposé durant le TME1 et aussi proposé dans l'article et que nous souhaitons tout de même implémenter. L'intuition est de se dire qu'en partant des noeuds qui possèdent le plus de voisins et donc qui ont les degrés les plus élevés, cela permettrait de faire A+- d'être plus facilement évaluable car on s'assure de partir de noeuds qui possèdent le plus de successeurs.

Notre deuxième intuition était de partir sur le taux d'homophilie d'un graphe. Il repose sur la similarité entre les nœuds. Autrement dit, si deux nœuds présentent de nombreuses caractéristiques communes (ou des structures voisines similaires), la probabilité qu'un lien existe entre eux est élevée. En nous appuyant sur ce principe, rappelons qu'un triangle dans un graphe orienté correspond à un triplet de nœud $\{u, v, w\}$ tels qu'il existe au moins un cycle ou ensemble de trois arêtes reliant ces nœuds entre eux, selon les directions.

Dans cette logique, à partir d'un nœud v , si l'on considère deux de ses successeurs et que ceux-ci présentent un fort taux de similarité, il est probable qu'ils forment un triangle avec v . Notre heuristique se base donc sur cette observation : pour chaque nœud, nous calculons la moyenne des taux de similarité entre ses successeurs. Plus un nœud possède des successeurs similaires entre eux, plus la probabilité qu'il soit impliqué dans un triangle est élevée.

Enfin, une dernière heuristique, dérivée de la notion de degré d'un nœud, nous est venue : le classement des nœuds selon leur coefficient de clustering local (LCC).

Le LCC mesure, pour un nœud donné u , la proportion de ses successeurs qui sont interconnectés entre eux, par rapport au nombre total de connexions possibles entre ces successeurs.

En d'autres termes, plus le LCC d'un nœud est élevé, plus ses successeurs forment un voisinage dense. Dans le contexte de la variante A+-, cette propriété est particulièrement intéressante : si un nœud possède un LCC important, la probabilité de former un triangle orienté à partir de ce nœud augmente, car les successeurs ont davantage de chances d'être reliés entre eux — ce qui facilite la validation du marquage au sein de l'algorithme.

De plus, un triangle orienté peut être vu comme une petite communauté locale dans le graphe, ce qui renforce l'intuition selon laquelle un fort coefficient de clustering est un indicateur pertinent pour prioriser les nœuds dans l'ordonnement.

3. Heuristiques implémentées

L'heuristique sur le classement par degré ne sera pas présentée car il figure déjà dans l'article. Nous porterons plutôt de l'attention sur des heuristiques qui diffèrent.

a. Ordonnement par la somme de degré des successeurs

i. Description

Suite à l'intuition sur la similarité, nous nous sommes penchés sur faire une équivalence pour le degré.

Le but étant de trier les nœuds par leurs taux de degré successeur.

Explication : Prenons deux nœuds de fort degré, la probabilité qu'il soit lié d'une quelconque manière est forte. Donc suivant le même principe que la similarité, si on prend un nœud v . Si ses successeurs ont un degré fort alors la probabilité d'avoir des triangles est plus élevée, car ils auront des arêtes sortantes et entrantes importantes.

Pour cela l'idée est simple :

- pour chaque nœud v du graphe :
 - on calcule la moyenne des degrés des successeurs
 - on trie ensuite les nœuds par moyenne du plus élevé au moins élevé

Pseudo code :

```
Fonction succ_degree( $G$ )  
   $order := []$   
  Pour chaque  $v$  de  $G$  :  
     $sum := 0$   
    Pour chaque  $u$  de  $Succ(v)$  :  
       $sum := sum + degree(u)$   
    Fin pour  
     $moy := sum / longueur(Succ(v))$   
    Ajout de  $v$  dans  $order$  (en fonction de  $moy$ )  
  Fin pour  
   $order := Sorted(order)$  (ordre décroissant)  
  Retourner  $order$   
Fin Fonction
```

ii. Complexité

Soit $n = |V|$ le nombre de nœuds dans le graphe, $m = |E|$ le nombre d'arêtes et $d(v)$ le degré d'un nœud v .

Dans le pire des cas :

- Pour chaque nœud du graphe : $O(n)$
- Pour toutes les successeurs d : $O(d(v))$
- Sorted : $O(n \cdot \log(n))$

Cela équivaut à un complexité dans le pire des cas à :

$$O((\sum_{v \in |V|} d(v)) + n \cdot \log(n)) = O(m + n \cdot \log(n))$$

b. Ordonnancement par la similarité

i. Description

Comme mentionné dans la partie précédente, cette heuristique a pour objectif de mesurer le taux moyen de similarité entre les successeurs d'un nœud.

Les graphes utilisés dans notre étude sont composés de nœuds abstraits, identifiés uniquement par un identifiant unique, sans attributs supplémentaires (tels que des informations sur des entités réelles). Ainsi, la similarité entre deux nœuds ne dépend que de leur degré dans le graphe.

Pour cette raison, nous évaluons la similarité entre deux nœuds à partir de leurs ensembles de successeurs respectifs, en utilisant le coefficient de Jaccard.

Pour cela l'idée est simple :

- pour chaque noeud v du graphe :
 - on calcule le taux de similarité entre pour chaque paire (u, w) avec $u, w \in \{Succ(v)\}$ et $u \neq w$
 - on fait la moyenne de ces taux pour avoir le taux moyen
- on trie ensuite les noeuds par taux du plus élevé au moins élevé

Pseudo code :

```

Fonction similarity(G)
  order := []
  Pour chaque v de G :
    sum := 0
    pairs := 0
    Pour chaque u de Succ(v) :
      Pour chaque w de Succ(v) : (avec u ≠ w)
        sum := sum + Jaccard(u,w)
        pairs := pairs + 1
      Fin pour
    Fin pour
    taux := sum / pairs
    Ajout de v dans order (en fonction de taux)
  Fin pour
  order := Sorted(order) (ordre décroissant)
  Retourner order
Fin Fonction

```

ii. Complexité

Soit $n = |V|$ le nombre de noeuds dans le graphe, $m = |E|$ le nombre d'arêtes et $d(v)$ le degré d'un noeud v .

Dans le pire des cas :

- Pour chaque noeud du graphe : $O(n)$

- Pour toutes les paires $(u, w) \in \{Succ(v)\} : O(d(v^2))$
- Coefficient de Jaccard : $O(\min(d(u) + d(w)))$ (utilisation de set)
- Sorted : $O(n \cdot \log(n))$

Cela équivaut à une complexité dans le pire des cas à :

$$\left(\sum_{v \in |V|} (d(v^2) + \min(d(u) + d(w))) \right) + n \cdot \log(n) \subseteq O(n^4)$$

c. Ordonnement par clustering

i. Description

D'après notre intuition, nous souhaitons évaluer le LCC revisité des nœuds. Soit $n = |V|$ le nombre de nœuds dans le graphe, $m = |E|$ le nombre d'arêtes et $d(v)$ le degré d'un nœud v .

Dans le pire des cas : pour les classer car l'algorithme A+ repose sur le fait d'aller repérer les successeurs communs de u et v , avec v un successeur de u . Pour rappel, le classement se fait par nombre d'interconnexions existant des successeurs de u par rapport au nombre total d'interconnexions possibles des successeurs.

Pseudo-code:

```

fonction LCC(noeud) :
    successeurs := successeurs_du_noeud(noeud)
    degre_sortant := nombre_de_successeurs(noeud)
    si degre_sortant < 2 alors
        retourner 0.0
    fin si
    aretes_entre_successeurs := 0
    pour chaque v dans successeurs faire
        successeurs_v := successeurs_du_noeud(v)
        aretes_entre_successeurs := aretes_entre_successeurs +
                                taille(successeurs_v ∩ successeurs)
    fin pour
    coefficient := aretes_entre_successeurs / (degre_sortant × (degre_sortant - 1))
    retourner coefficient
fin fonction

```

Il suffit d'itérer ce coefficient sur chacun des nœuds du graphe, les stocker dans un dictionnaire associant le nœud à son LCC et de retourner la liste des nœuds dont on ordonne de façon décroissante par la valeur du LCC.

ii. Complexité

Soit n le nombre de nœud du graphe, V l'ensemble des nœud du graphe et $d^+(u)$ l'ensemble des successeurs de u . Nous savons que nous allons devoir itérer sur tous les nœuds ce qui est en $O(n)$. Ensuite, pour la complexité de LCC, récupérer les successeurs se fait en $O(1)$, même chose pour le degré sortant. L'opération la plus coûteuse est l'intersection entre les successeurs de v et de u . L'intersection se fait sur des sets, et par définition, l'intersection de deux sets est:

$$O(\min(d^+(u), d^+(v)))$$

La boucle concerne fait la somme des aretes des successeurs. On a donc avec u le noeud courant

$$\sum_{v \in d^+(u)} O(\min(d^+(u), d^+(v)))$$

Ce qui est au plus $O((d^+(u))^2)$ si on suppose que v et u sont de même grandeur.

Si on itère cela sur tous les noeuds du graphe, on arrive à la complexité suivante:

$$O(\sum_{u \in V} (d^+(u))^2)$$

Etudes expérimentales

La dernière partie de ce travail consiste à comparer expérimentalement les heuristiques proposées à l'ordonnement par dégénérescence, sur différents types de graphes orientés.

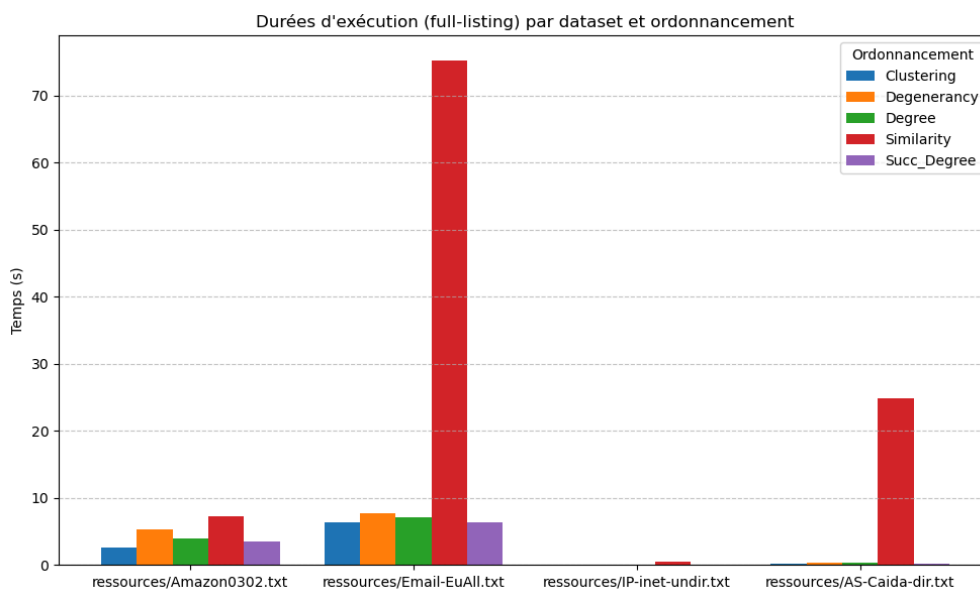
Pour cela, nous avons utilisé les graphes fournis lors du TME 1, ainsi que deux graphes supplémentaires issus de jeux de données réels : un graphe Amazon et un graphe d'échanges d'e-mails.

Cette étude repose sur deux types de comparaison :

- une comparaison classique des temps d'exécution,
- une évaluation du speed-up des heuristiques par rapport à l'ordonnement par dégénérescence.

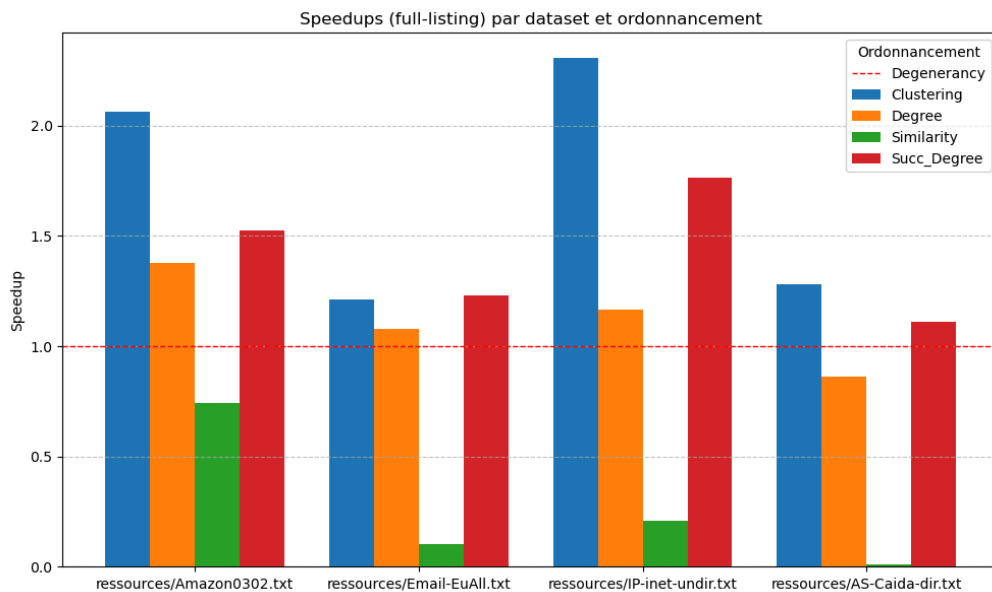
Les heuristiques sont ainsi évaluées selon deux modes de calcul : le mere-listing (*listing uniquement des triangles en fonction des ordonnancements*) et le full-listing (*exécution des heuristiques ainsi que du listing de triangles*)

Dans un premier temps, intéressons-nous aux résultats obtenus pour le full listing.



On remarque qu’une des heuristiques, celle basée sur la similarité, présente un temps d’exécution nettement plus élevé. Cela s’explique par sa complexité algorithmique importante, de l’ordre de $O(n^4)$ dans le pire cas, en raison du calcul systématique des similarités entre successeurs pour chaque nœud.

En revanche, les heuristiques de clustering et de degré des successeurs se révèlent plus efficaces que l’ordonnement par dégénérescence.



Cette amélioration est particulièrement visible sur le diagramme de speed-up des heuristiques présenté ci-dessous.

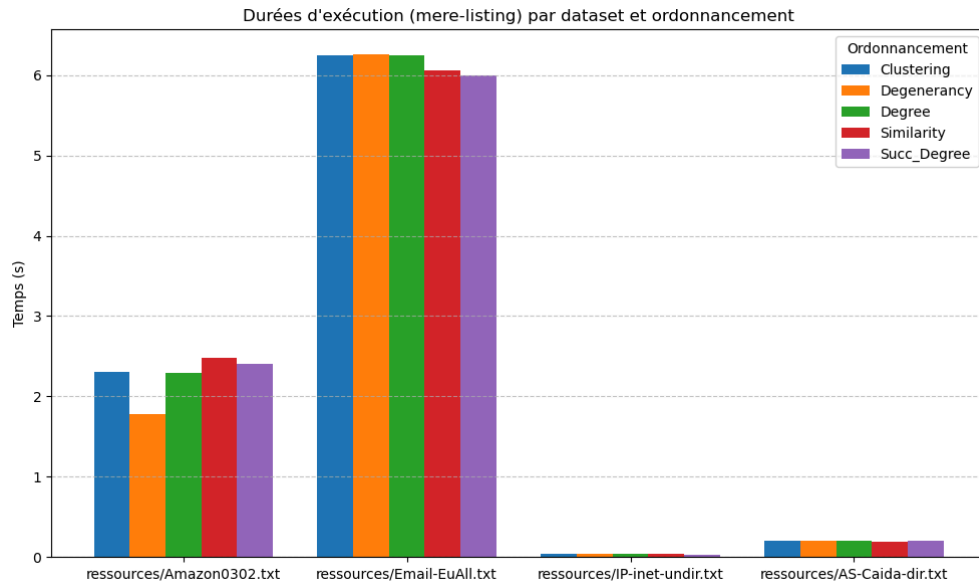
On y observe notamment que l’heuristique de clustering offre un gain de performance significatif par rapport à la dégénérescence.

Cependant, pour un graphe peu dense, cette même heuristique montre une légère baisse d’efficacité, la structure du graphe limitant l’intérêt du regroupement local des sommets.

Nous constatons également que l’heuristique du degré des successeurs apporte une amélioration modérée mais constante par rapport au classement par degré, confirmée expérimentalement.

En revanche, l’heuristique de similarité reste particulièrement lente pour le listing complet des triangles, ce qui la rend peu adaptée à ce type de traitement.

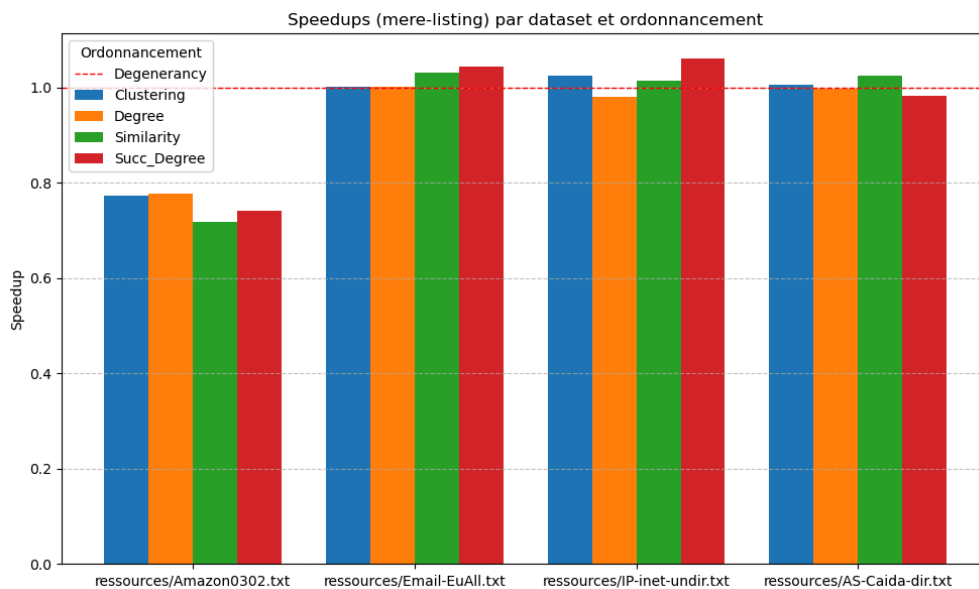
Enfin, il est intéressant de noter une amélioration ponctuelle sur le graphe Amazon, probablement liée à son taux d’homophilie et à sa structure hiérarchique, qui limitent le nombre de combinaisons de successeurs à analyser.



Concernant le mere-listing, nous observons que la plupart des ordonnancements présentent des performances relativement similaires pour l'énumération des triangles.

Toutefois, l'heuristique basée sur la dégénérescence se distingue par une meilleure efficacité sur l'un des graphes testés.

Globalement, le diagramme de speed-up met en évidence une légère amélioration apportée par l'heuristique du degré des successeurs et l'heuristique de similarité.



Cependant, pour confirmer ces observations et dégager des tendances plus robustes, il serait nécessaire de disposer d'un échantillon plus large et plus diversifié d'exécutions, afin d'évaluer précisément les performances réelles de chaque heuristique par rapport à l'ordonnancement par dégénérescence.

Enfin, notons que l'intuition de l'heuristique basée sur la similarité n'était pas mauvaise pour le mere-listing : elle repose sur une logique pertinente.

Néanmoins, en raison de sa complexité algorithmique élevée, elle demeure peu viable en pratique pour ce type d'application.

Conclusion

Dans ce rapport, nous avons étudié l'ordonnancement des triangles dans des graphes orientés et proposé plusieurs heuristiques basées sur le degré des nœuds, le clustering local et la similarité. L'objectif était de réduire le temps de recherche par rapport à l'ordonnancement par dégénérescence.

Nos expérimentations montrent que :

- L'heuristique du degré des successeurs apporte une amélioration notable et constante.
- Le clustering local permet de repérer efficacement les nœuds formant des triangles, surtout dans des graphes avec des groupes locaux denses.
- L'heuristique par similarité, bien que logique, est trop lente pour des graphes de grande taille en raison de sa complexité élevée.
- L'ordonnancement par dégénérescence reste un point de référence solide et efficace.

Ces résultats confirment que l'efficacité d'une heuristique dépend fortement de la structure du graphe. Pour aller plus loin, il serait intéressant de tester ces méthodes sur plus de graphes variés et d'explorer des approches hybrides combinant plusieurs heuristiques.

** ChatGpt a été utilisé pour la formulation de phrases et certains correctifs de code et de texte*